Computer Architecture
(Performance, CPU, ISA)

Dept. Applied AI, SeoulTech

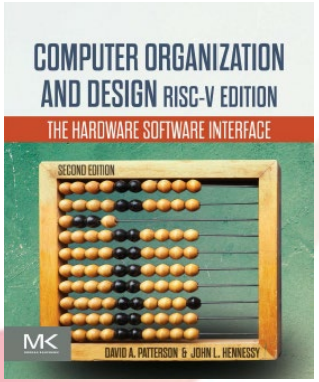
Beomseok Oh

Contents

- Performances
- CPU and its components
- Registers
- Arithmetic and Logic operation Unit (ALU)
- Instruction set architecture (ISA): CISC vs. RISC
- Classifying instruction set architectures

Performances

(ch. 1.6, COD book)

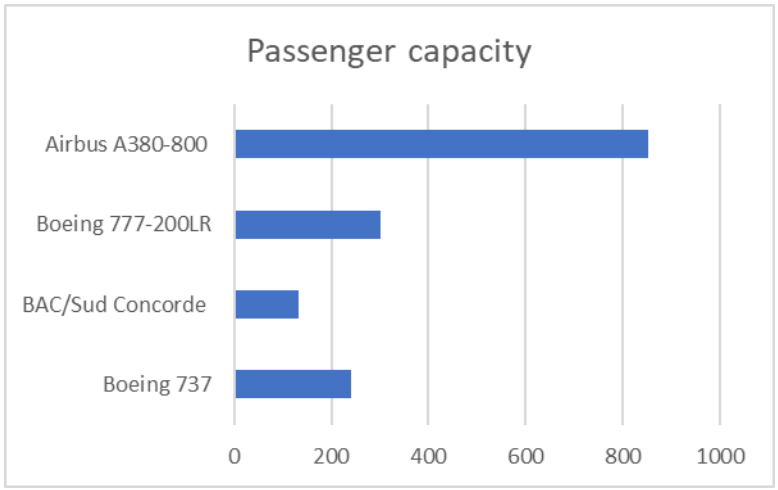


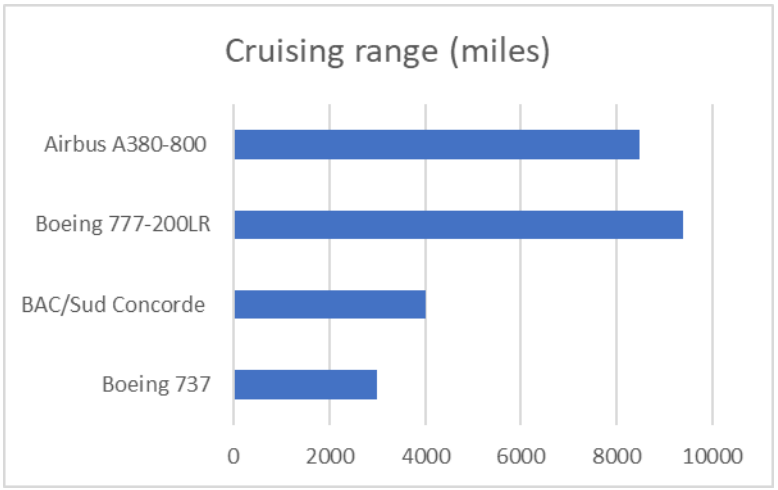
Understanding Performance

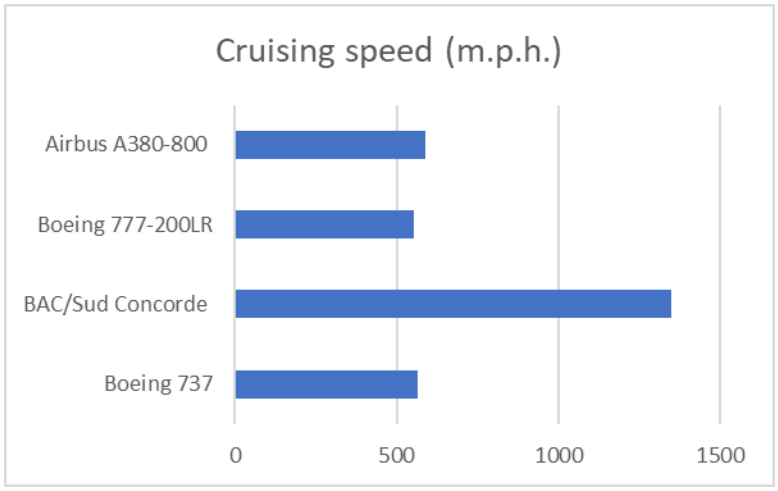
- Algorithm
  - Determines number of operations executed
- Programming language, compiler, architecture
  - Determine number of machine instructions executed per operation
- Processor and memory system
  - Determine how fast instructions are executed
- I/O system (including OS)
  - Determines how fast I/O operations are executed

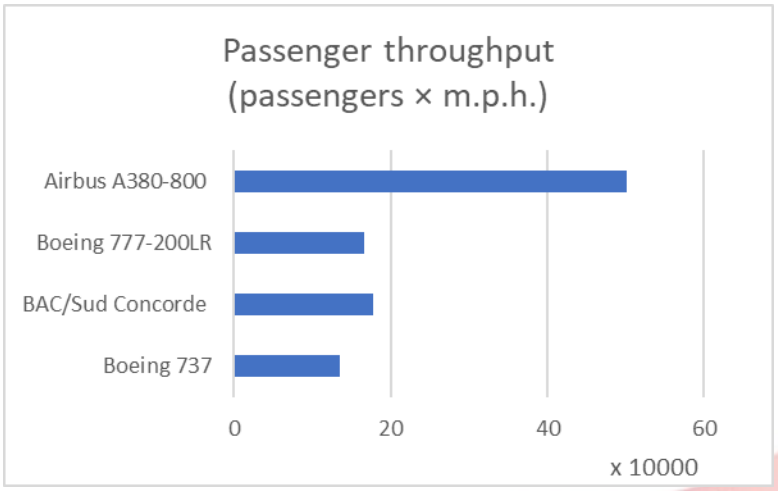
Defining performance

- Which airplane has the best performance?









Response Time and Throughput

- Must have a clear definition of what performance metric matters
- Response time
  - a.k.a. execution time, wall clock time, elapsed time
  - Total time to complete a task including disk access, memory access, I/O activities, OS overhead
  - Usually measured in seconds
- Throughput
  - Total work done per unit time
    - e.g., tasks/transactions/… per hour
- System may try to optimize throughput rather than minimizing the elapsed time for a specific program
  - Computers are often shared
  - A processor may work on several programs simultaneously

Response Time and Throughput – cont'd

- How are response time and throughput affected by
  - Replacing the processor with a faster version?
  - Adding more processors?
- We'll focus on response time for now…

Relative Performance

- Definition

$$\text{Performance} = \frac{1}{\text{Execution time}}$$

- Assume computers X and Y. If the performance of X is greater than that of Y, we have

$$\text{Performance}_X > \text{Performance}_Y$$

$$= \frac{1}{\text{Execution time}_X} > \frac{1}{\text{Execution time}_Y}$$

$$= \text{Execution time}_Y > \text{Execution time}_X$$

- "X is n times faster than Y"

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

Relative Performance – Cont'd

- Example: time taken to run a program
  - If 10s on A, and 15s on B, then

$$\frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{15s}{10s} = 1.5$$

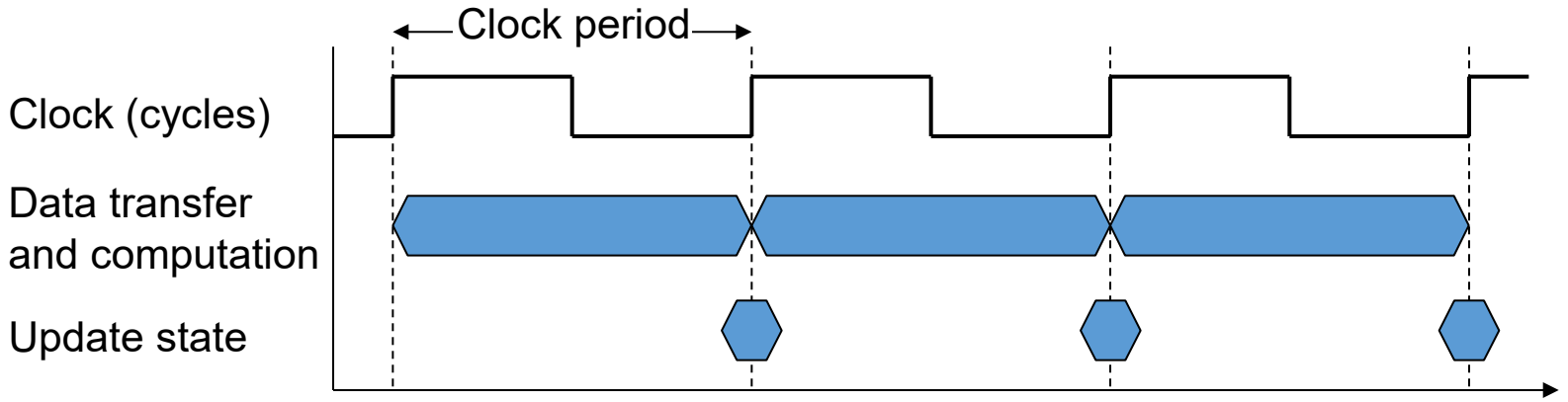
  - So, A is 1.5 times faster than B

Measuring performance

- Time is the measure of computer performance
- Elapsed time
  - Total response time, including all aspects
    - Processing, I/O, OS overhead, idle time
  - Determines system performance
- CPU time (a.k.a. CPU execution time)
  - Time spent processing a given job
    - Discounts I/O time, other jobs' shares
  - Comprises user CPU time and system CPU time
  - Different programs are affected differently by CPU and system performance
- Another metric, clock cycles per instruction (CPI) might be useful
  - Clock cycle – discrete time intervals
    - a.k.a. ticks, clocks, cycles, clock periods
  - CPI – average number of clock cycles to complete an instruction

CPU Clocking

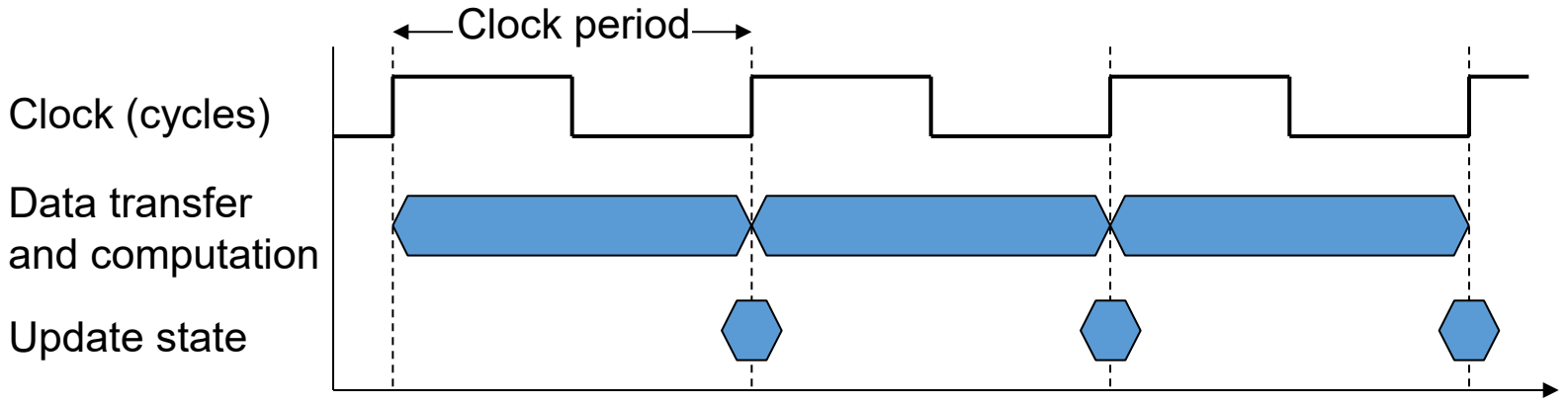
- Operation of digital hardware governed by a constant-rate clock



- Clock pulse (from Wikipedia)
  - In electronics and especially synchronous digital circuits, a clock signal (historically also known as logic beat) is an electronic logic signal (voltage or current) which oscillates between a high and a low state at a constant frequency and is used like a metronome to synchronize actions of digital circuits. In a synchronous logic circuit, the most common type of digital circuit, the clock signal is applied to all storage devices, flip-flops and latches, and causes them all to change state simultaneously, preventing race conditions.

CPU Clocking – Cont'd

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
  - e.g., 250ps = 0.25ns = 250×10^{-12}s
- Clock frequency (rate): cycles per second
  - e.g., 4.0GHz = 4000MHz = 4.0×10^{9}Hz
  - Inverse of the clock period

CPU Time

- Performance improved by
  - Reducing the number of clock cycles
  - Increasing clock rate
  - Hardware designer must often trade off clock rate against cycle count

$$\text{CPU execution time for a program} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

$$\text{CPU Clock Cycles} = \text{CPU Clock Cycles for a program}$$

CPU time example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
  - Aim for 6s CPU time
  - Can do faster clock, but causes 1.2 times as many clock cycles as Com. A
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6\text{s}}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10\text{s} \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4\text{GHz}$$

Instruction Count and CPI

- Instruction count for a program
  - Determined by program, ISA and compiler
- Average cycles per instruction
  - Determined by CPU hardware
  - If different instructions have different CPI
    - Average CPI affected by instruction mix

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$$

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$CPU\ time_A = \text{Instruction Count} \times CPI_A \times Cycle\ time_A$$

$$= I \times 2.0 \times 250ps = I \times 500ps$$

A is faster…

$$CPU\ time_B = \text{Instruction Count} \times CPI_B \times Cycle\ time_B$$

$$= I \times 1.2 \times 500ps = I \times 600ps$$

$$\frac{CPU\ time_B}{CPU\ time_A} = \frac{I \times 600ps}{I \times 500ps} = 1.2$$

…by this much

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^{n} \left(\underbrace{\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency}} \right)$$

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for each class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

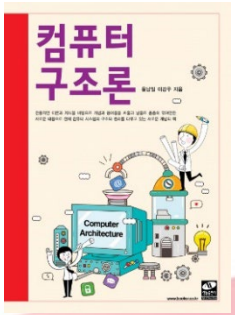
- Sequence 1: IC = 5
  - Clock Cycles
    = 2×1 + 1×2 + 2×3 = 10
  - Avg. CPI = 10/5 = 2.0
- Sequence 2: IC = 6
  - Clock Cycles
    = 4×1 + 1×2 + 1×3 = 9
  - Avg. CPI = 9/6 = 1.5

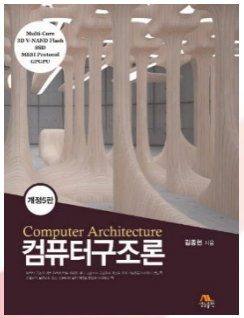
Performance Summary

- Performance depends on
  - Algorithm: affects IC, possibly CPI
  - Programming language: affects IC, CPI
  - Compiler: affects IC, CPI
  - Instruction set architecture: affects IC, CPI, T_c

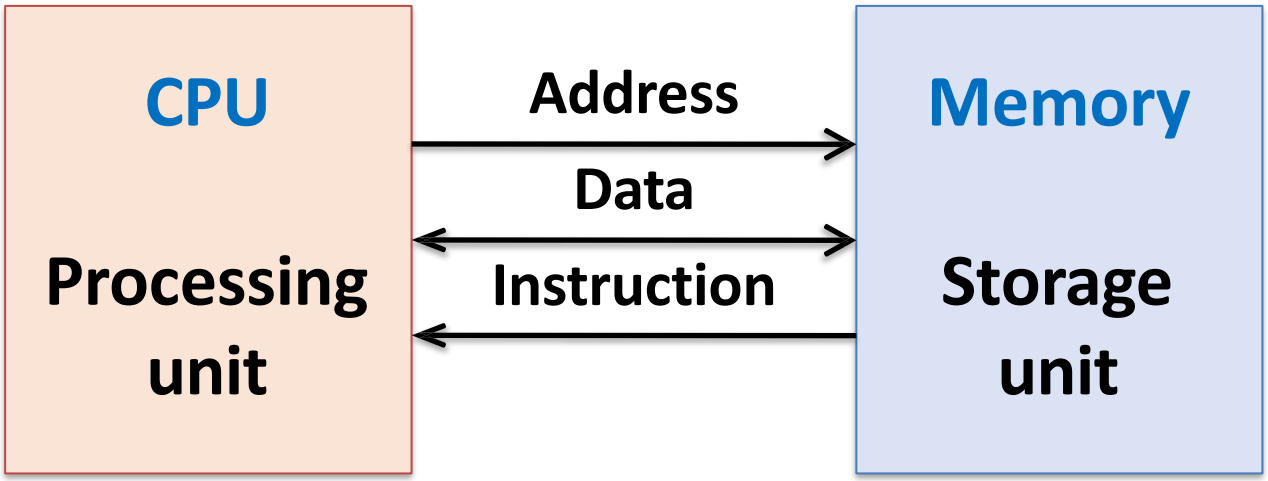
$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

CPU and its components





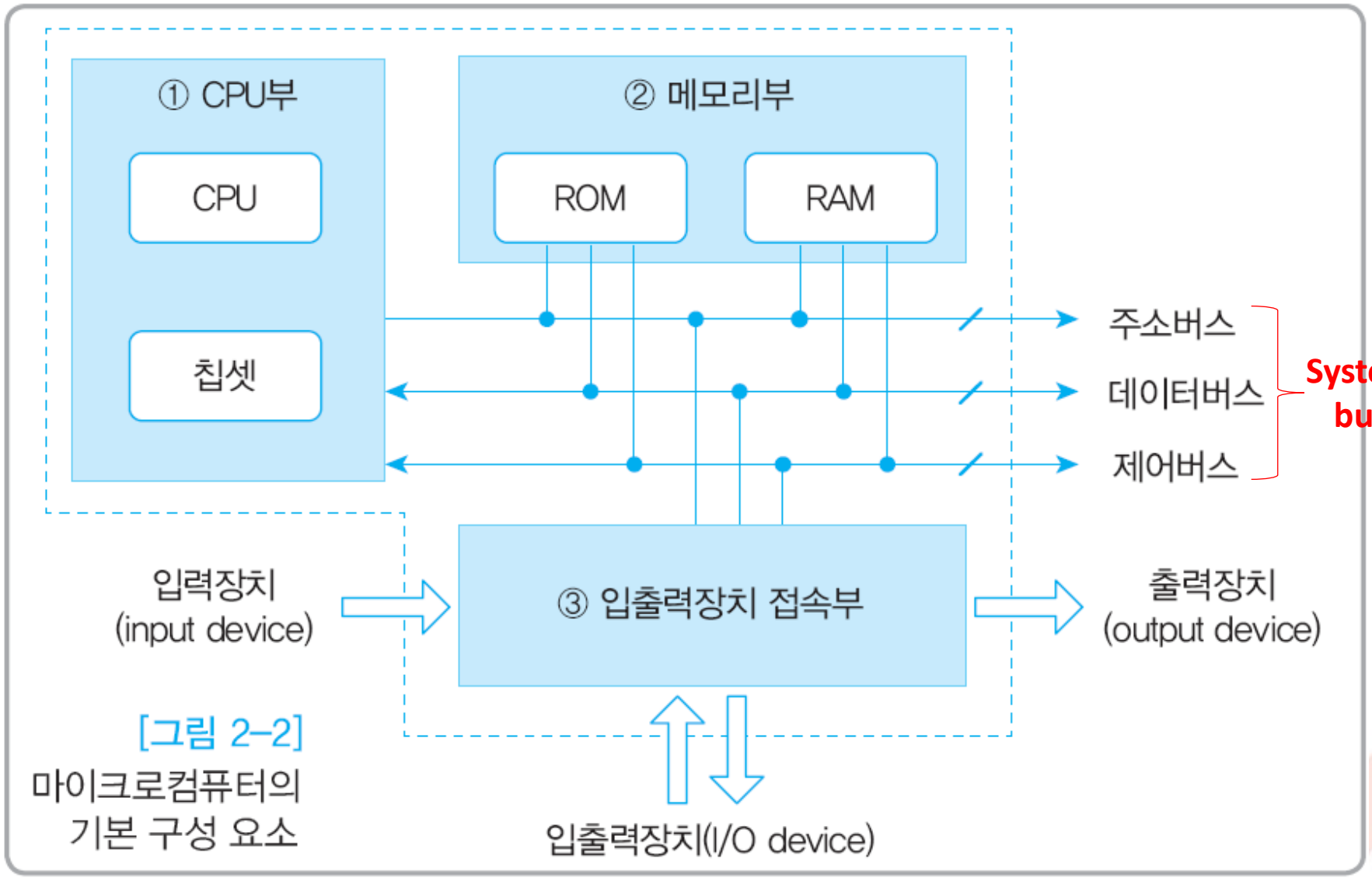
CPU, memory and I/O



Data movement
Arithmetic & logical ops
Control transfer

Byte addressable array
Code + data (user program, OS)
Stack to support procedures

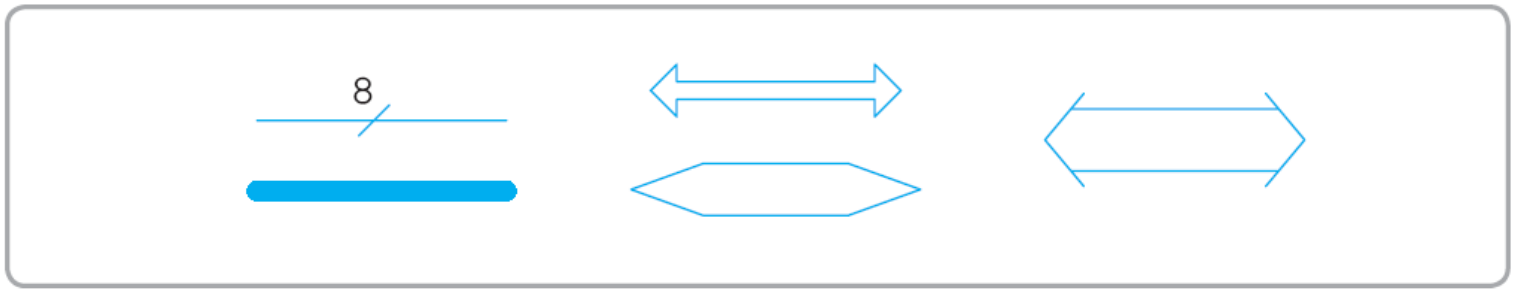
CPU, memory and I/O – Cont'd



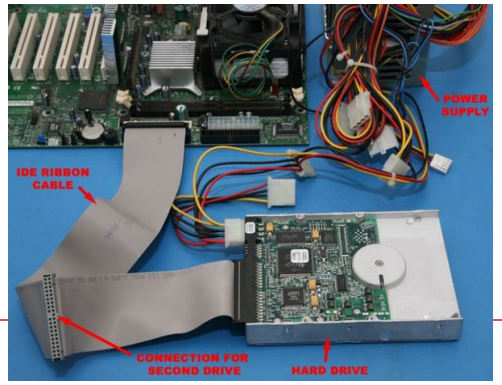
[그림 2-2]
마이크로컴퓨터의
기본 구성 요소

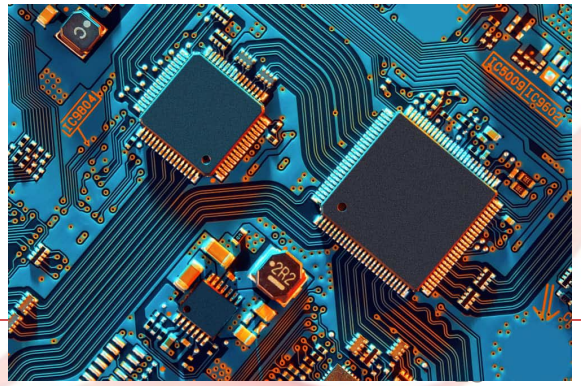
Bus

- 정보를 교환하기 위해 CPU와 하드웨어 요소들을 연결해주는 신호선들의 집합
  - 정보는 0 또는 1로 구성된 2진 데이터
  - 전자회로 부품을 연결하는 신호선의 다발
  - 하드웨어로 설치된 전선(hard-wired)들의 집합



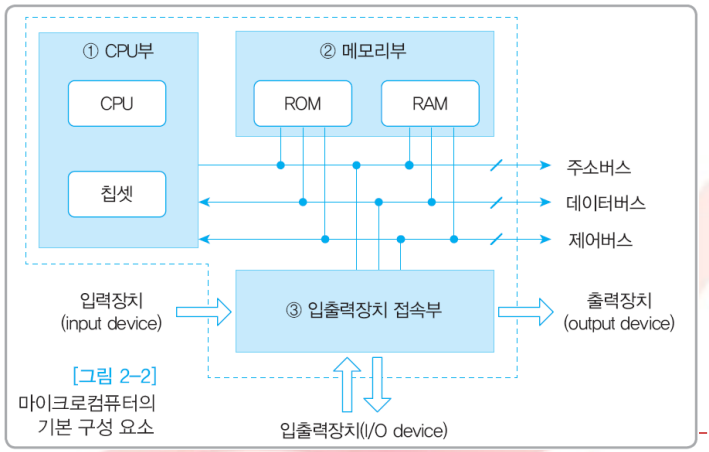
[그림 2–4] 버스선의 다양한 표시 방법





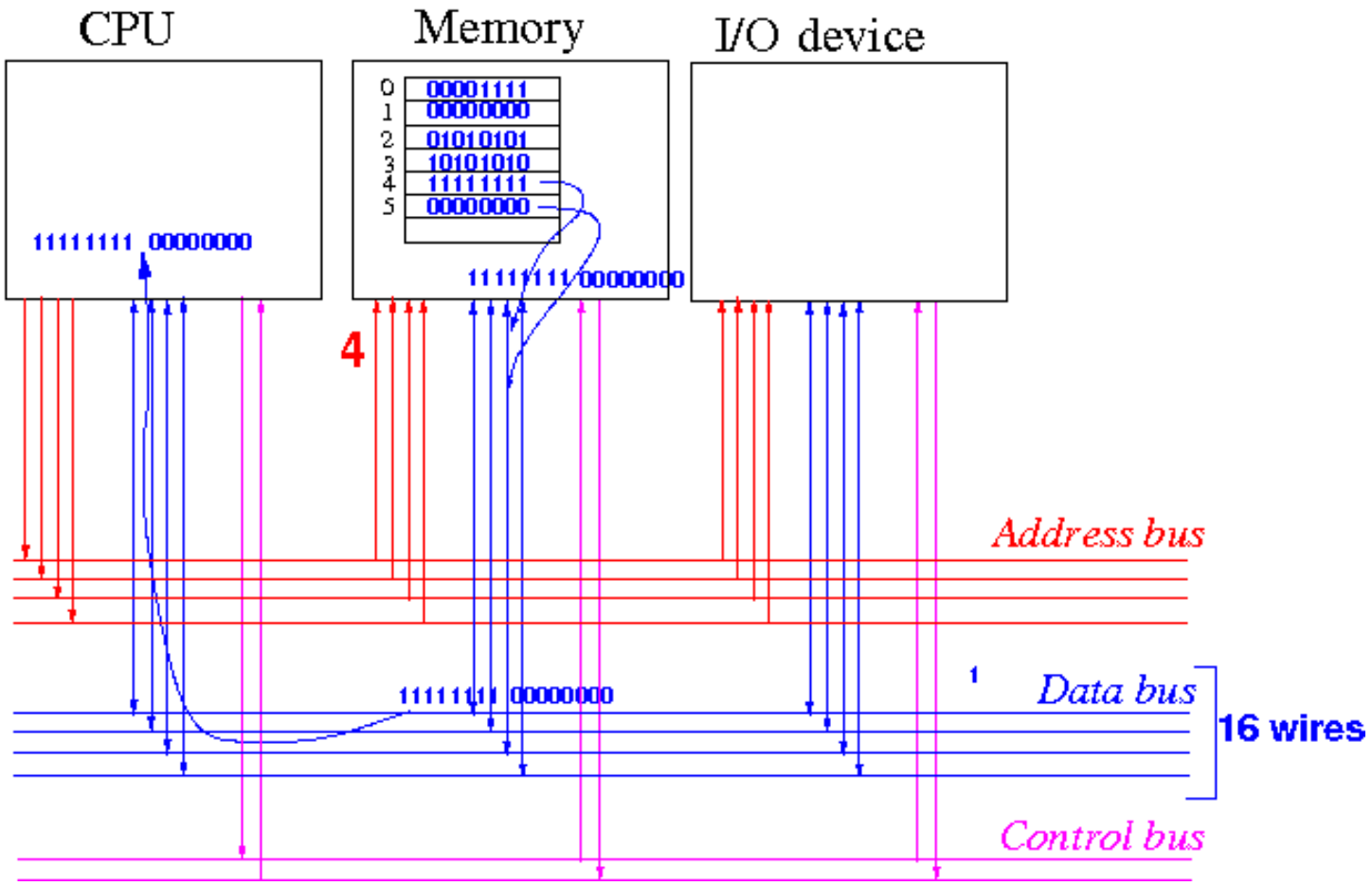
Bus의 종류

- Address bus
  - CPU가 외부로 내보내는 주소 신호, 단방향 전송
  - Address bus 배선 수 → 외부 기억장치의 최대 용량 결정
  - Address bus의 width가 16bit이면 최대 $2^{16} = 65{,}536$개 기억 장소들의 주소 지정 가능
- Data bus
  - CPU가 메모리나 I/O 장치와 데이터를 주고 받는 통로
  - CPU가 한 번에 전송할 수 있는 비트 수, 양방향 전송
  - Data bus의 width가 32bit이면 한 번에 32bit씩 전송 가능
- Control bus
  - CPU 내외부의 장치를 동작시키는 제어신호
    - 기억장치 읽기와 쓰기
    - 입출력장치 읽기와 쓰기 신호 등
  - 단방향 또는 양방향 전송



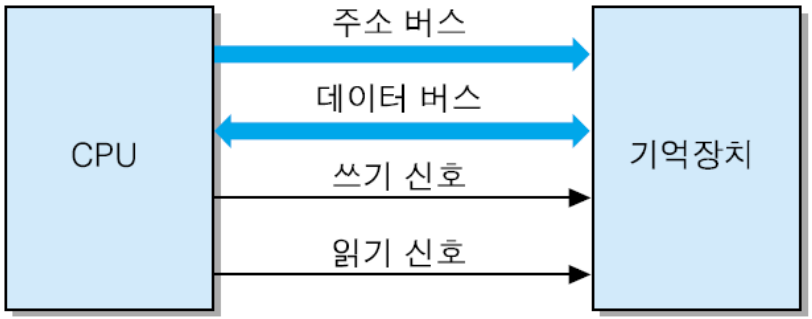
[그림 2-2]
마이크로컴퓨터의
기본 구성 요소

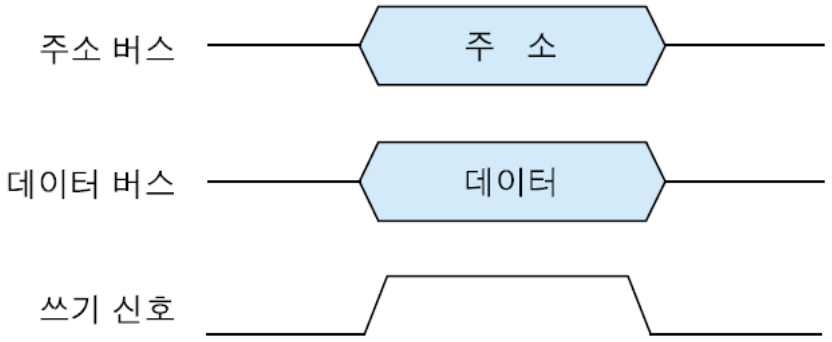
Bus의 종류 – Cont'd



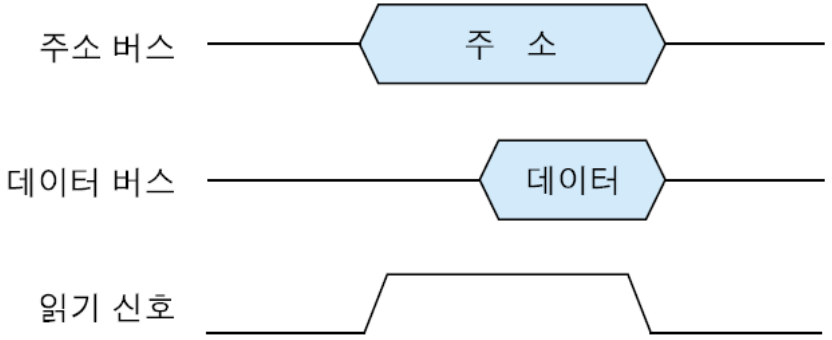
Bus의 동작 원리

- E.g., CPU – RAM 간의 읽기, 쓰기 동작





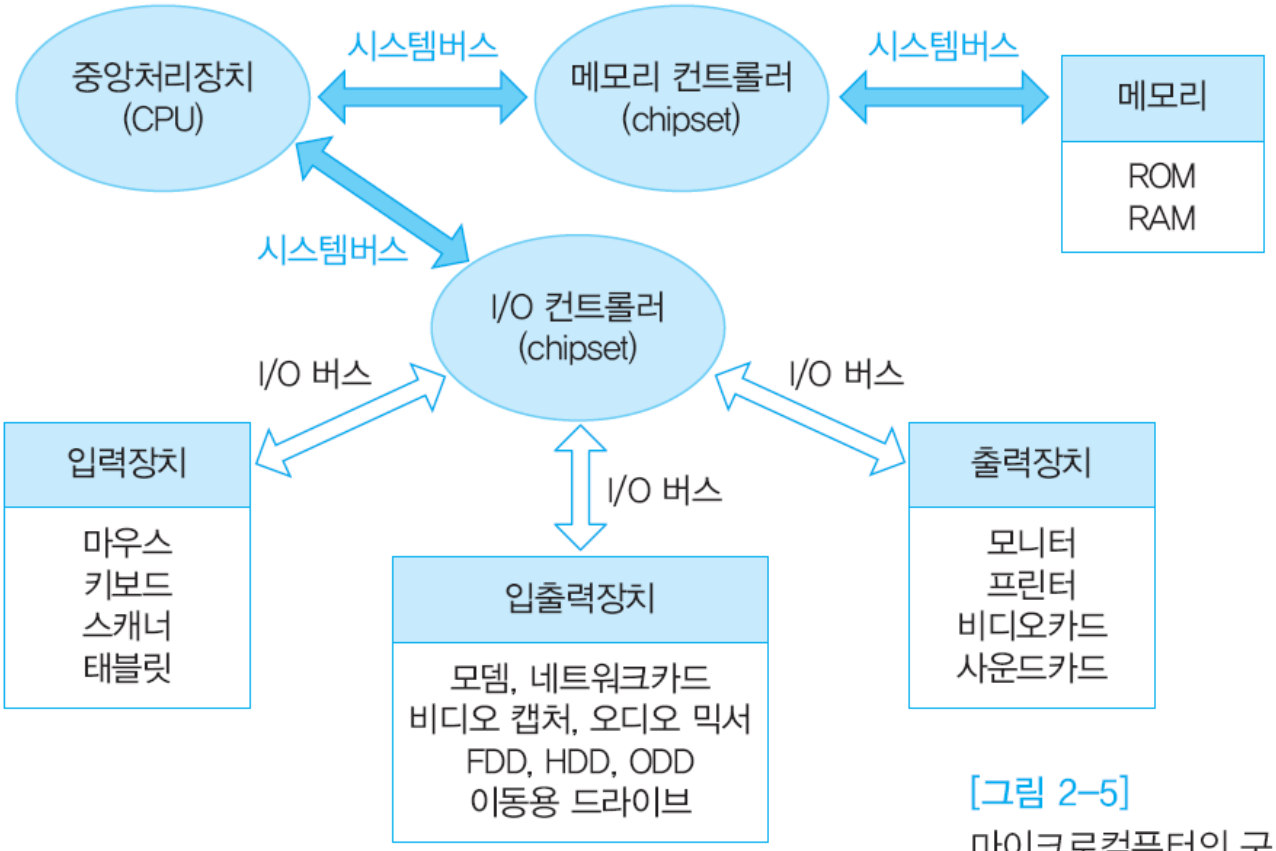
(a) 기억장치 쓰기 동작의 시간 흐름도



(b) 기억장치 읽기 동작의 시간 흐름도

Computer 구조

- 서버 등 상위 기종은 CPU 성능을 높이고 고속의 소자를 사용
  - 다중 프로세서 구조로 공통의 시스템버스와 I/O 버스
  - 에러검출 및 정정기능이 강화된 고가격 소자들 사용



[그림 2-5]
마이크로컴퓨터의 구조

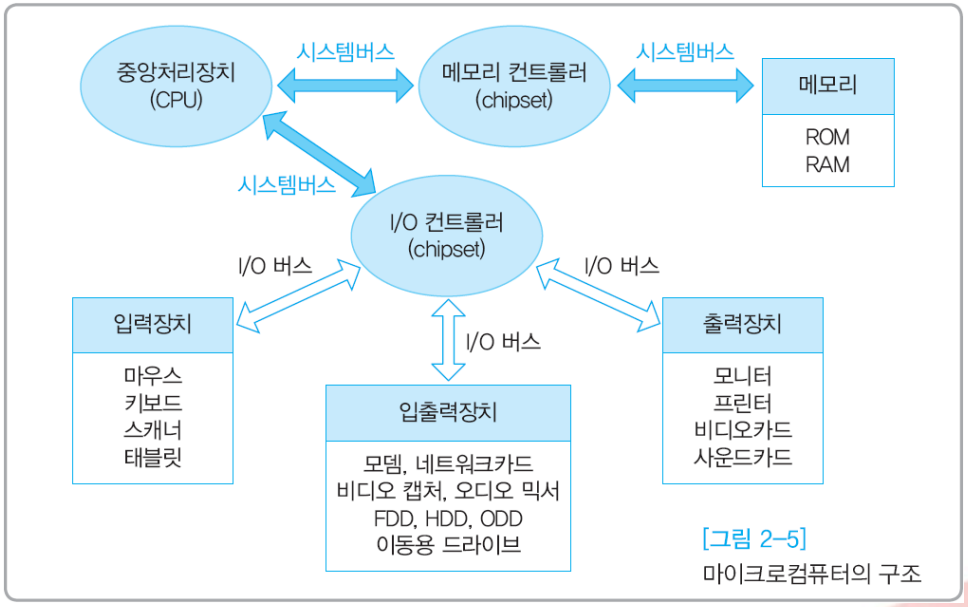
Address와 controller

- Memory address와 I/O address
  - 하드웨어 측면에서 memory와 I/O 구분
  - Memory address
    - 주로 반도체 memory: RAM
  - I/O address
    - 나머지 장치들
    - HDD는 보조기억장치이나 하드웨어적으로 I/O 장치

- Memory controller와 I/O controller
  - CPU를 도와 하드웨어를 제어, 대개 chipset으로 구성
  - Memory controller
    - 메모리 장치를 제어
  - I/O controller
    - 입출력장치를 제어

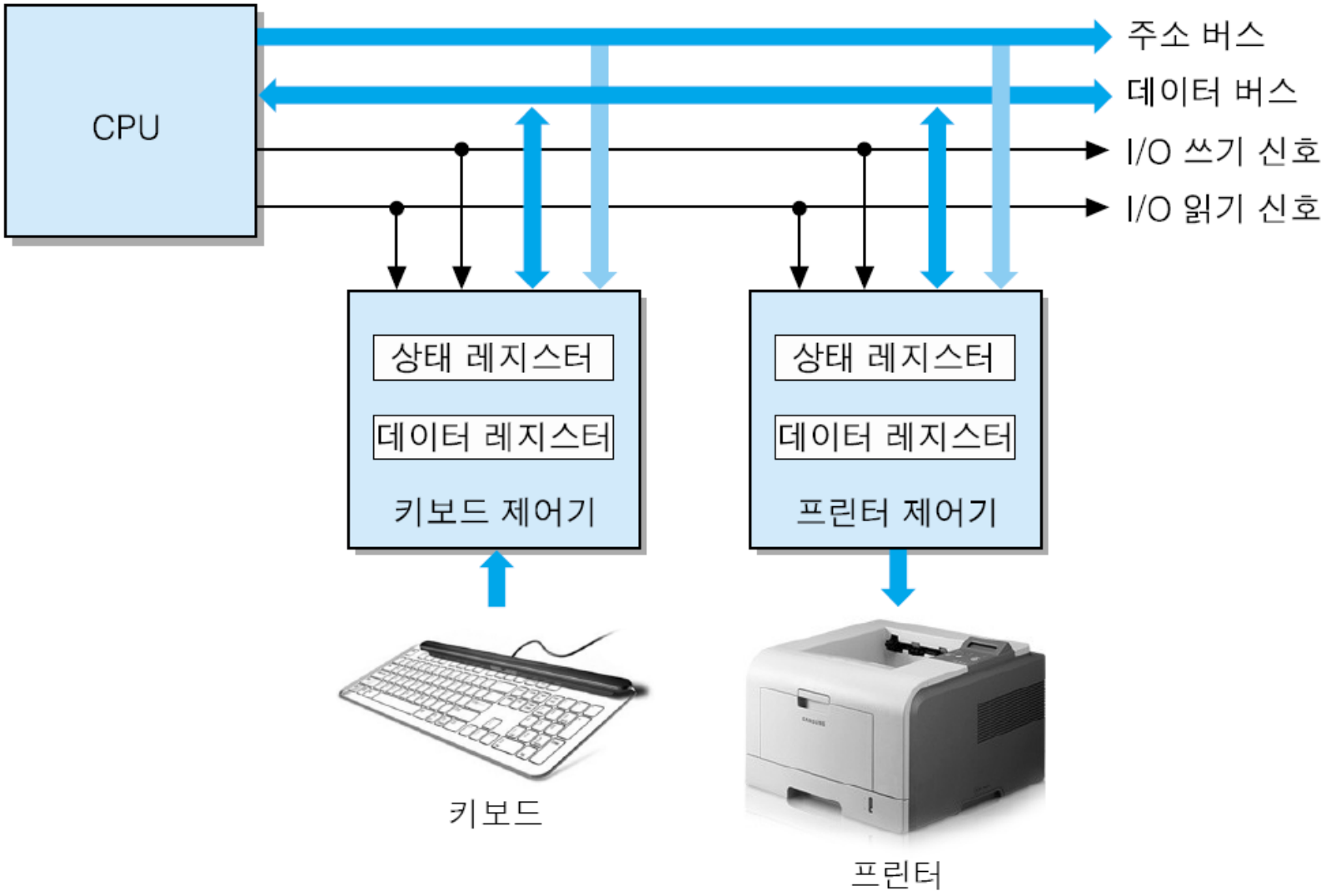
System bus와 I/O bus

- System bus
  - CPU, main memory, chipset들과 연결
  - RAM 모듈을 CPU에 동기시킴 (제어시간 절약)
- I/O bus
  - I/O controller와 입출력장치 사이를 연결
  - 비트 수와 속도가 다양한 여러 종류의 I/O 버스 사용



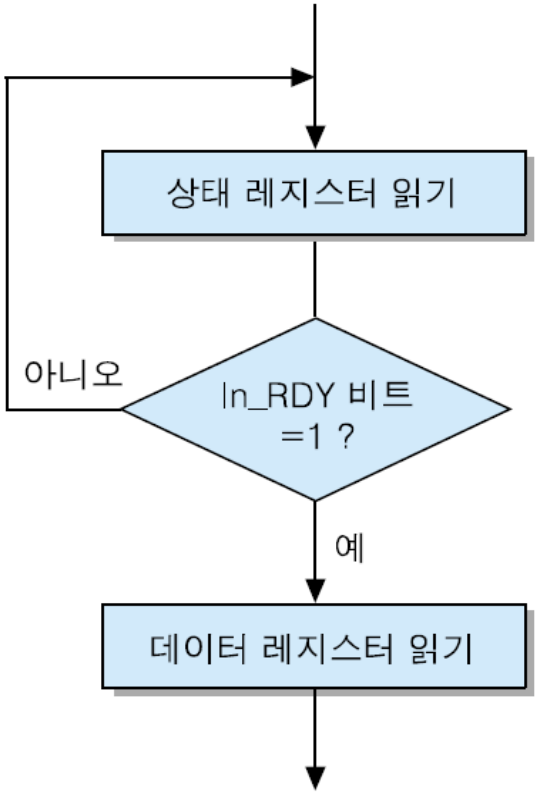
[그림 2-5]
마이크로컴퓨터의 구조

I/O bus 처리과정


CPU
주소 버스
데이터 버스
I/O 쓰기 신호
I/O 읽기 신호
상태 레지스터
데이터 레지스터
키보드 제어기
상태 레지스터
데이터 레지스터
프린터 제어기
키보드
프린터

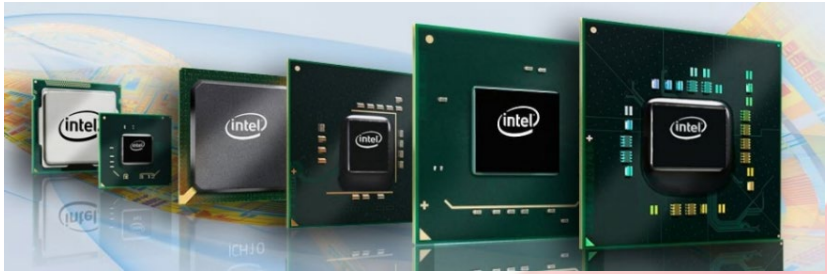
I/O bus 처리과정: 키보드 예

- 키보드
  - 한 key가 눌리면 해당되는 ASCII 코드가 데이터 레지스터에 저장
  - 동시에 상태 레지스터의 In_RDY 비트 1로 세트
- CPU
  - 키보드 제어기로부터 상태 레지스터의 내용 읽어서 In_RDY 세트 여부 검사
  - 만약 세트되지 않았다면 반복 대기
  - 만약 세트되었다면 데이터 레지스터 읽음



Chipset

- 여러 칩(chip) 과 회로가 모여 서로 연관된 기능을 수행하도록 설계된 제어 chip들의 조합(set)
  - 컴퓨터, 휴대전화, 디지털 TV 등 모든 전자제품에는 특수 목적용 chipset이 들어 있음
- CPU 프로세서와 함께 시스템 전체를 제어
  - 특정 CPU를 사용해 하드웨어를 구성하도록 설계
- 대개 mainboard 상에 몇 개의 제어 칩들로 구성
- Chipset 내부 회로
  - CPU를 지원하는 각종 제어 장치들
  - Bus controller, memory controller, I/O controller, interrupt controller, timer 등 다양한 장치 포함

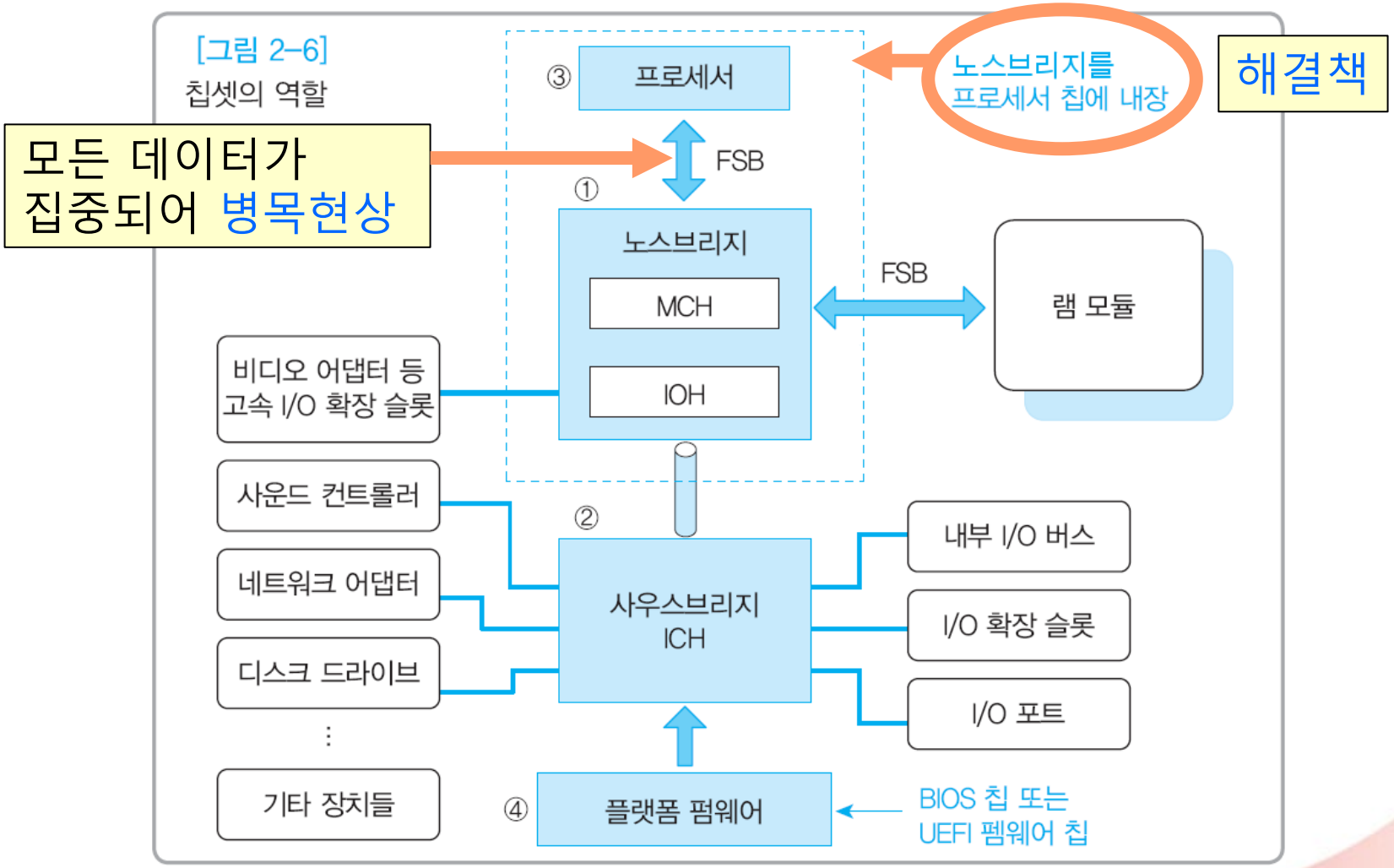


Chipset의 구성

- Memory controller 와 I/O controller가 기본 구성
  - North bridge
    - Memory 컨트롤러 포함, 주로 고속의 장치들
  - South bridge
    - I/O 컨트롤러 포함, 주로 저속의 장치들
  - Processor와 firmware 칩까지 한 구성으로 보기도

- 브리지, 허브
  - 칩셋이 “중심 연결 장치”라는 뜻
  - Memory controller hub (MCH)
  - I/O controller hub (IOH 혹은 ICH)

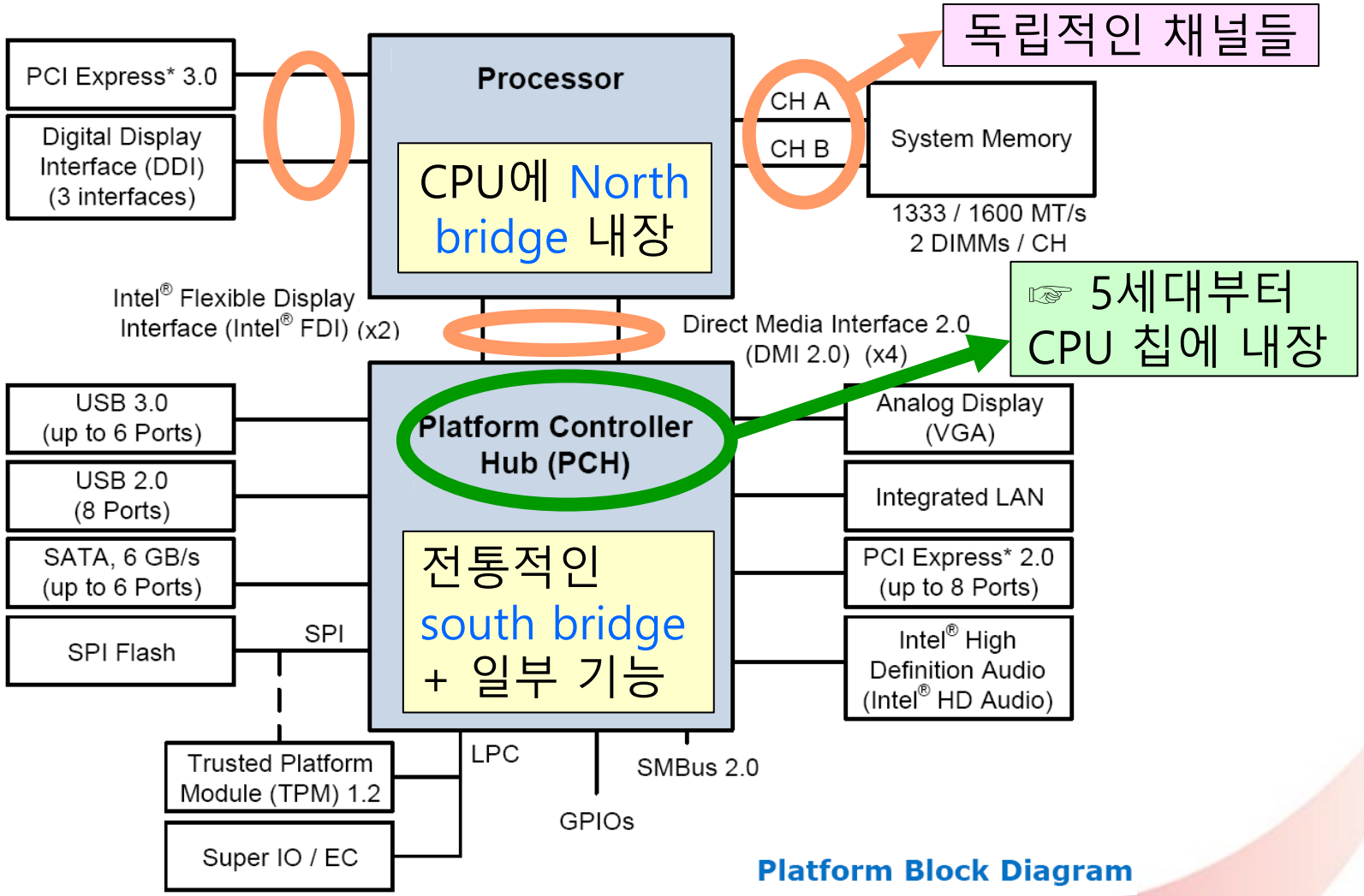
Chipset은 CPU 프로세서에 대한 플랫폼



Chipset의 구성 추세

- 비디오 어댑터 인터페이스를 내장
  - North bridge에 포함: 그래픽 전용 채널로 설계
  - 저가 설계의 Graphic processing unit (GPU)의 경우 CPU에 내장
- 프로세서 칩에 North bridge 내장
  - RAM 모듈, 비디오, South bridge를 독립채널로 분리
    - 데이터가 몰리는 front-side bus (FSB)의 병목현상 해결
    - RAM 모듈 접근속도 높여 멀티코어 프로세서 성능 향상
  - 전통적인 North bridge의 거의 모든 기능을 CPU 내장
    - E.g.) 인텔의 platform controller hub(PCH)
      - 전통적인 south bridge + north bridge의 일부 기능
- 프로세서 칩에 칩셋을 모두 내장

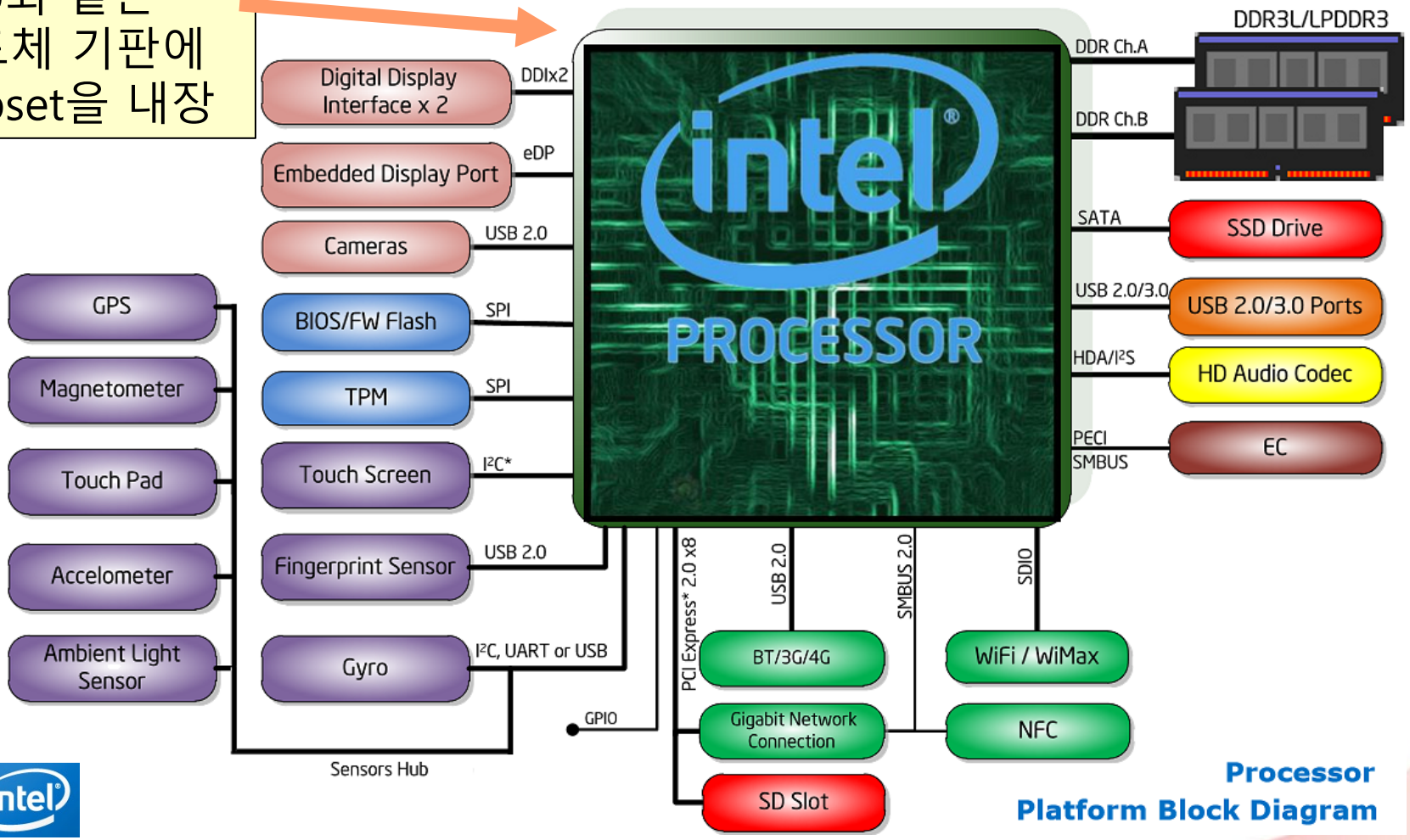
인텔 4세대 Core 프로세서 플랫폼 (2013.09)



Platform Block Diagram

인텔 5세대 Core 프로세서 플랫폼 (2015.01)

CPU와 같은
반도체 기판에
chipset을 내장



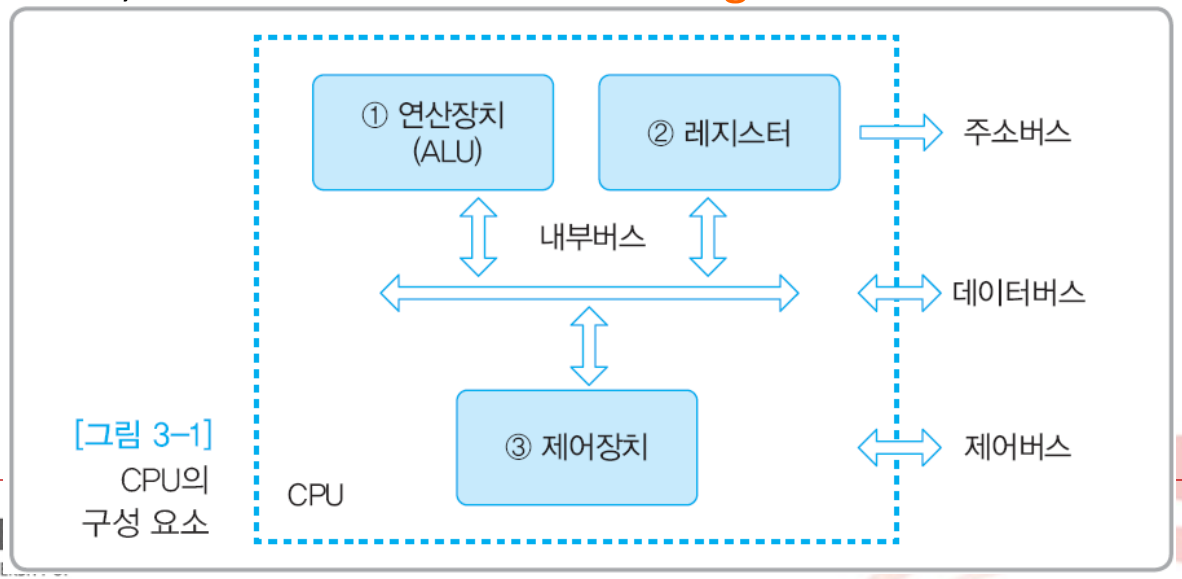
Processor Platform Block Diagram

Central processing unit (CPU)

- 명령어와 데이터를 처리
  - 프로그램에서 주어진 명령어를 인출해 해독
  - 명령에 따라 데이터를 인출해 연산을 수행하고 그 결과를 저장하거나 다른 장치로 전송
- Process
  - 연산이나 데이터를 처리하기 위해 현재 진행되는 작업이나 프로그램
- Processing
  - 작업의 처리, 처리과정
- Processor
  - 컴퓨터 내부에 포함된 하나의 module, CPU chip 혹은 컴퓨터 자체

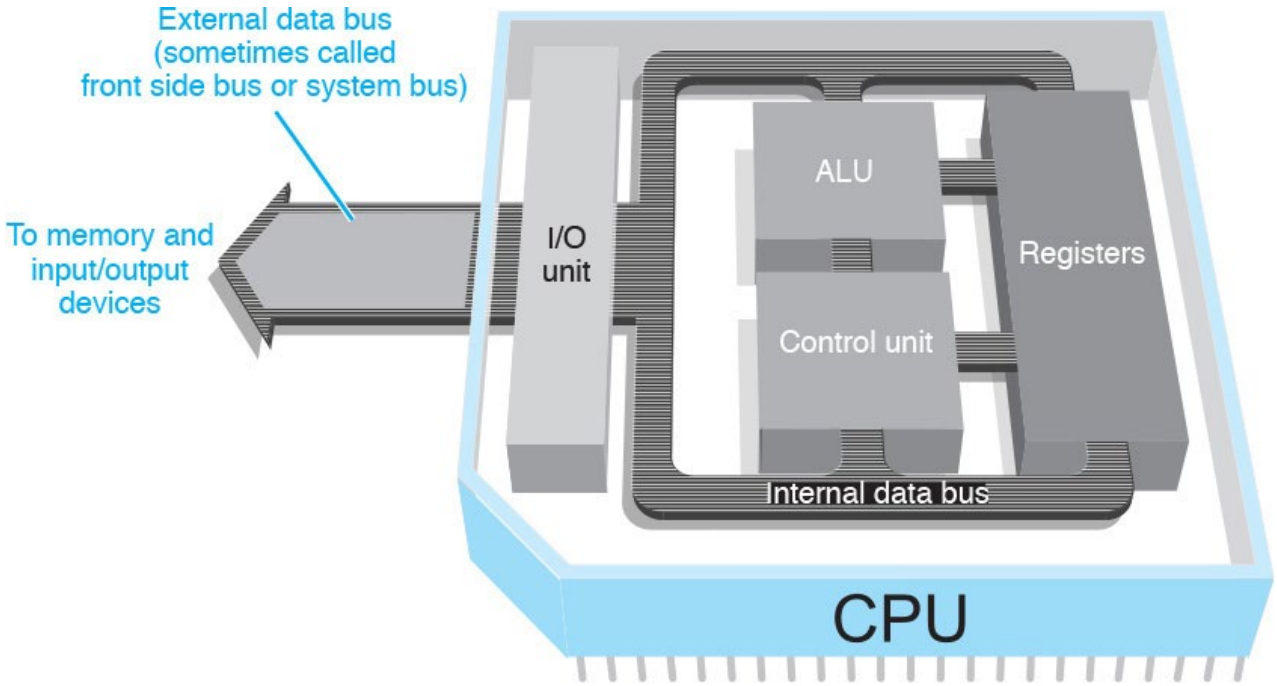
Central processing unit (CPU) 구성요소

- Arithmetic and logic operation unit (ALU)
  - 산술연산과 논리연산을 수행하는 하드웨어 부분
- Register
  - 연산을 위해 다양한 용도로 사용되는 CPU 내부의 일시적인 기억장소
  - 명령, 주소, 데이터 등을 일시 저장
- Control unit
  - 명령을 해석하고 실행하기 위한 control signal 발생
  - 연산, 읽기, 쓰기 등 동작신호와 timing 신호

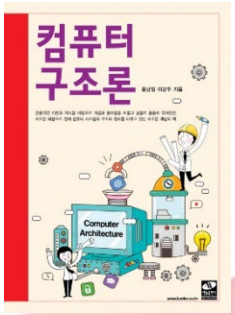


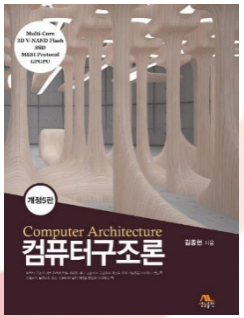
[그림 3-1] CPU의 구성 요소

Central processing unit (CPU) 구성요소



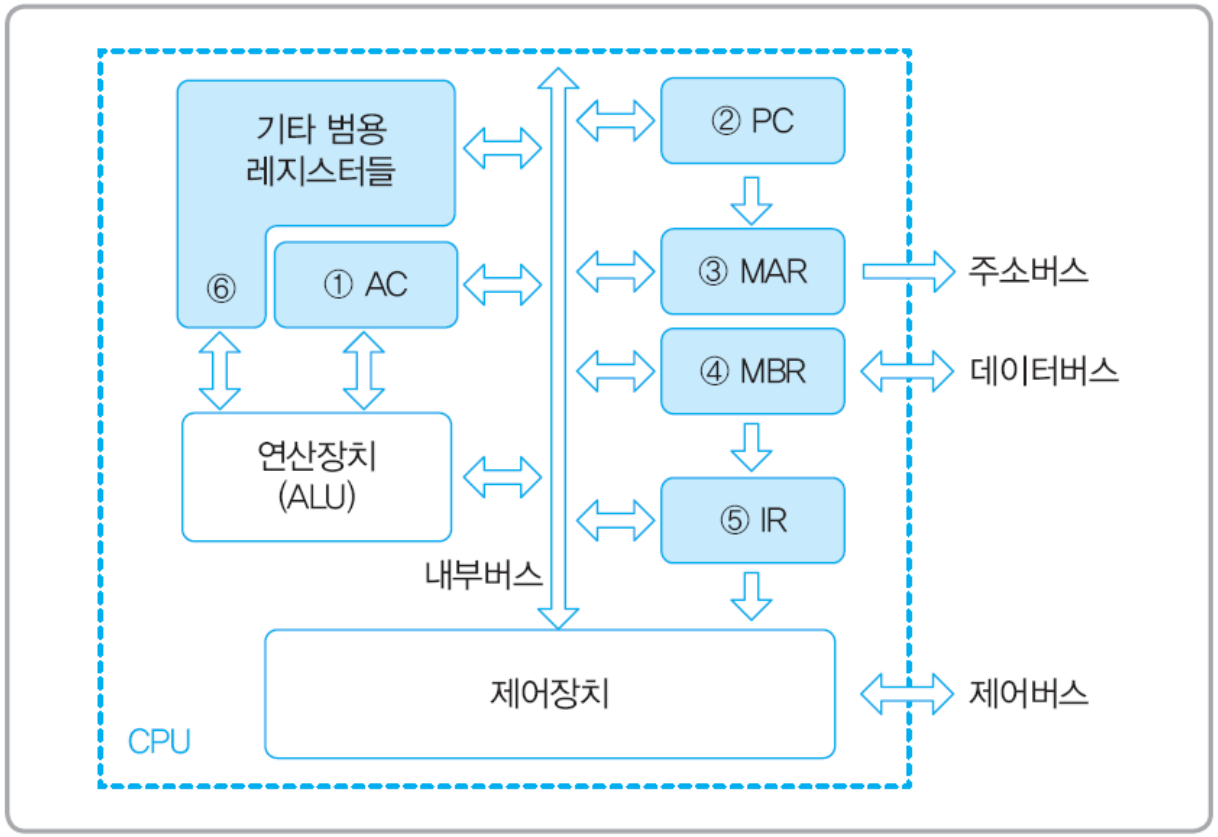
Register


컴퓨터
구조론
Computer
Architecture


Computer Architecture
컴퓨터구조론

Register의 구성

- Register
  - CPU 내부의 임시 저장장소



[그림 3-4] 레지스터의 기본 구성

AC (Accumulator)

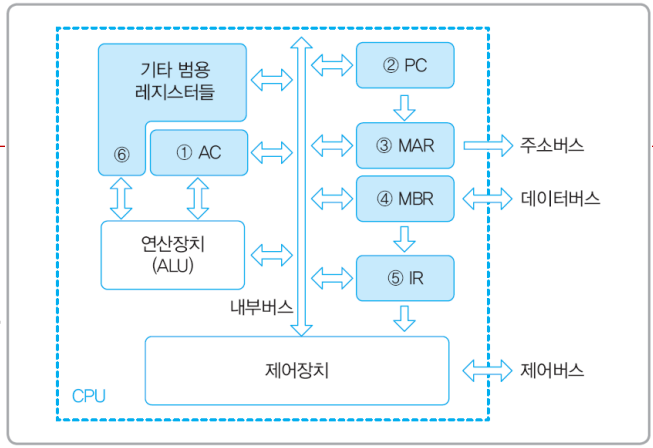
PC (Program counter)

MAR (Memory address register)

MBR (Memory buffer register)

IR (Instruction register)

Register의 구성



[그림 3-4] 레지스터의 기본 구성

- 몇 가지 가상의 register를 정의
  - Accumulator (AC) 혹은 누산기
    - 연산에 가장 빈번하게 사용, 데이터를 일시 저장
    - 연산 결과가 다시 자신에게 accumulation
  - Program counter (PC)
    - 다음에 인출해올 명령어의 주소를 저장
  - Memory address register (MAR)
    - Address bus로 주소를 출력하기 전에 주소를 임시 저장
  - Memory buffer register (MBR)
    - 데이터버스로 데이터를 읽고 쓸 때 임시로 저장
  - Instruction register (IR)
    - 가장 최근에 인출된 명령어를 저장
  - 기타 범용 register들
    - 연산 도중에 일시적인 기억장소로 사용
    - Accumulator : 데이터를 주로 저장
    - Pointer : 데이터의 주소를 저장
    - Counter : 데이터의 개수를 저장

- 실제 프로세서는 훨씬 많은 register 사용
- 각 레지스터에 붙이는 명칭들도 제조사마다 다름

Register의 구성 – Cont'd

- RISC-V architecture

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	–
x4	tp	Thread pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

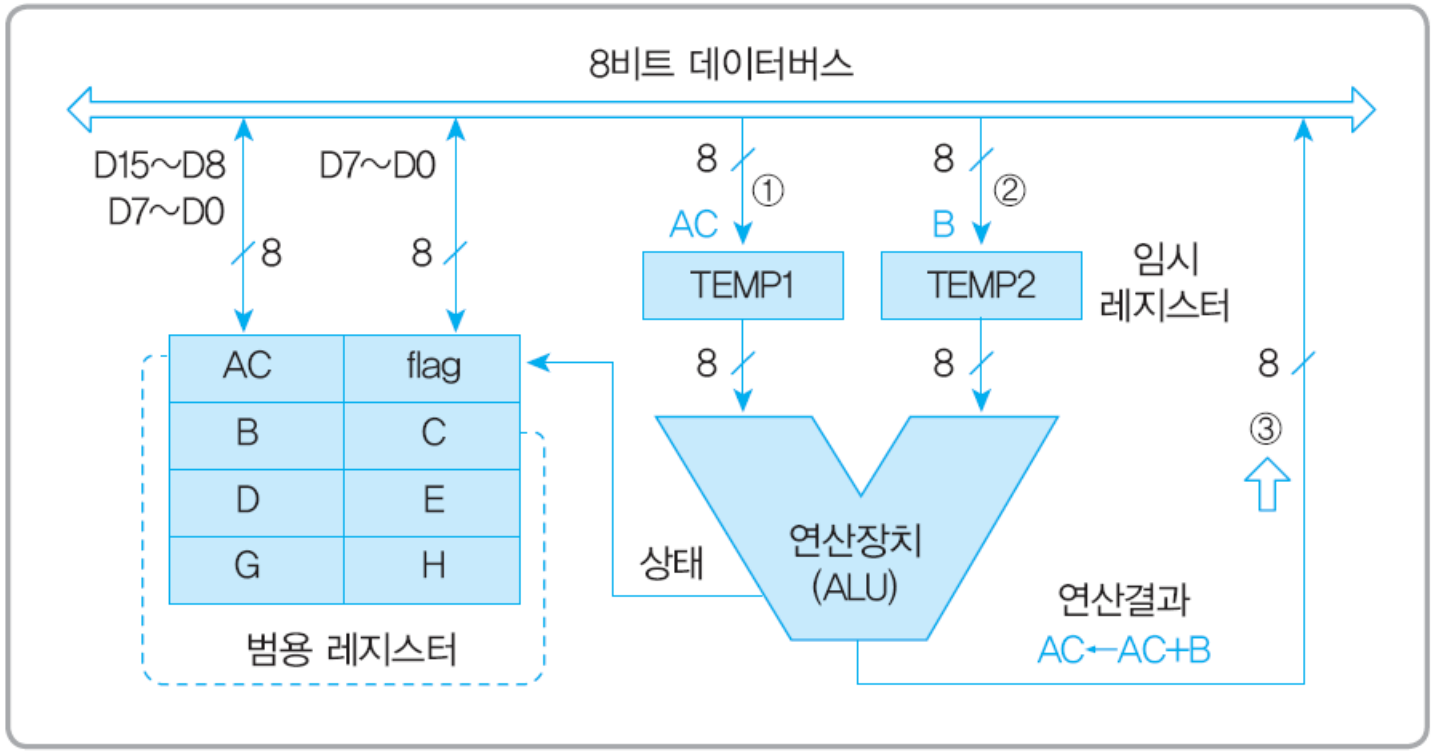
Arithmetic and Logic operation Unit (ALU)

ALU의 구성

- 수치 데이터에 대한 산술연산과 2진 데이터에 대한 논리연산이 이루어지는 부분

- 연산장치의 일반적인 구성 요소
  - Status register
    - 최종 실행된 연산결과의 상태를 저장
  - 2's complementer
    - 음수 연산을 위해 2의 보수를 생성
  - Arithmetic operator
    - 사칙연산(+,-,×,÷) 등 산술연산
  - Logic operator
    - NOT, AND, OR, XOR 등 논리연산
  - Shift register
    - 비트 열 이동

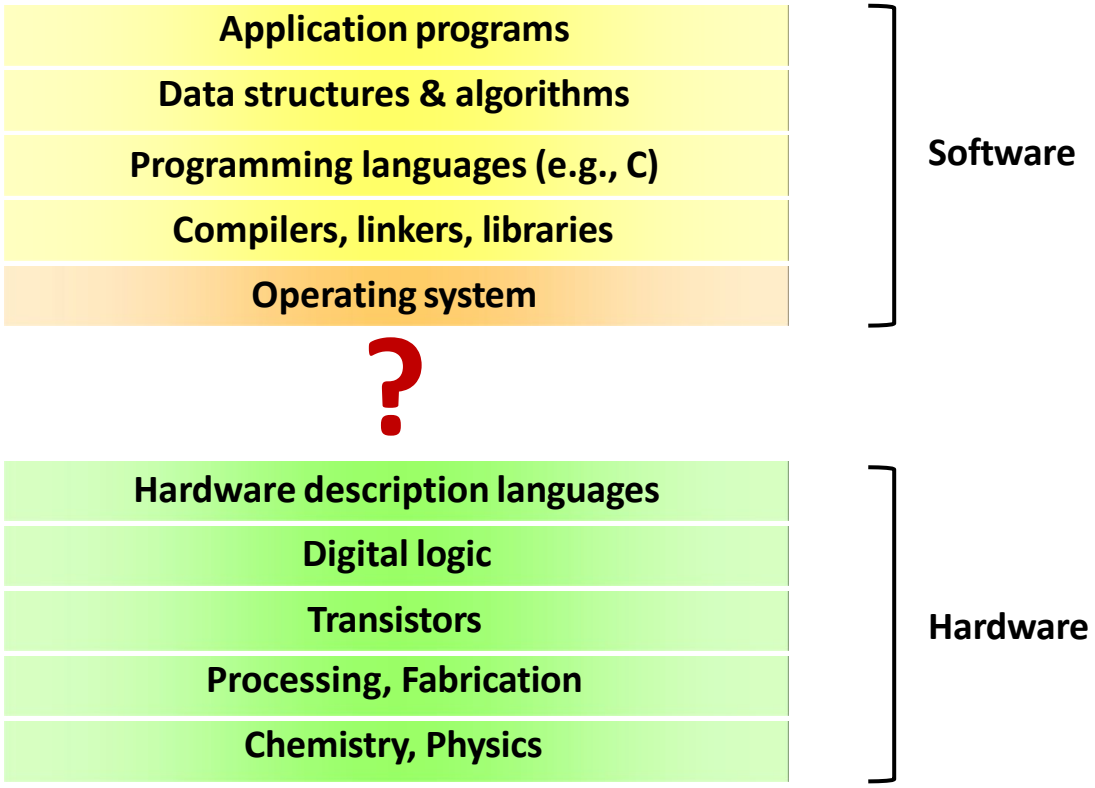
ALU의 연산 동작



[그림 3-5] 연산장치(ALU)의 기본 구조

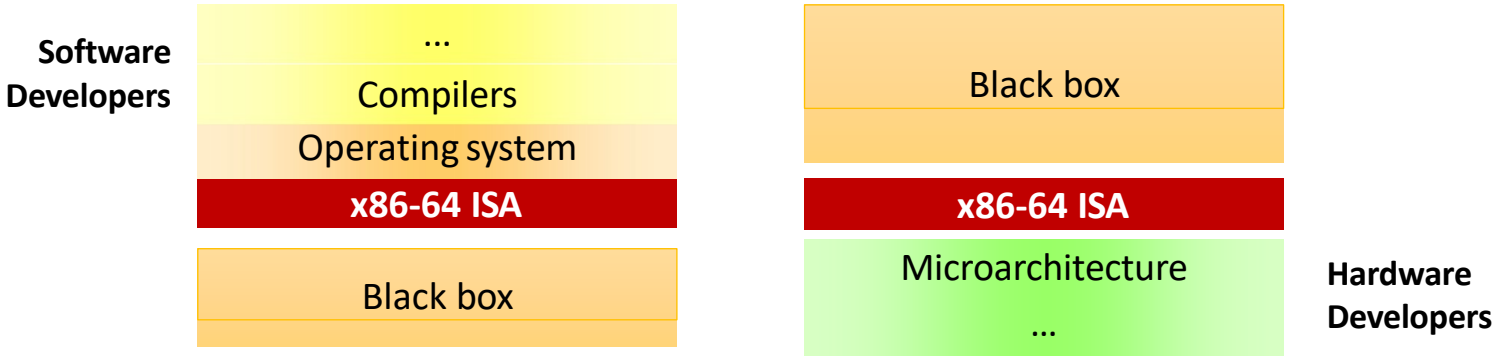
Instruction set architecture (ISA): CISC vs. RISC

How to run your program?



Instruction set architecture (ISA)

- The hardware/software interface
  - Hardware abstraction visible to software (OS, compilers, …)
  - Instructions and their encodings, registers, data types, addressing modes, etc.
  - Written documents about how the CPU behaves
  - e.g., All 64-bit Intel CPUs follow the same x86-64 (or Intel 64) ISA



Instruction Set Architecture (ISA) – Cont'd

- Class of ISA
  - General-purpose registers
  - Register-memory vs. load-store
- RISC-V registers
  - 32 g.p., 32 f.p.

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	–
x4	tp	Thread pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

Instruction Set Architecture (ISA) – Cont'd

- Memory addressing
  - RISC-V: byte addressed, aligned accesses faster
- Addressing modes
  - RISC-V: Register, immediate, displacement (base+offset)
  - Other examples: autoincrement, indexed, PC-relative
- Types and size of operands
  - RISC-V: 8-bit, 32-bit, 64-bit
- Operations
  - RISC-V: data transfer, arithmetic, logical, control, floating point
  - See Fig. 1.5 of CA book
- Control flow instructions
  - Use content of registers (RISC-V) vs. status bits (x86, ARMv7, ARMv8)
  - Return address in register (RISC-V, ARMv7, ARMv8) vs. on stack (x86)
- Encoding
  - Fixed (RISC-V, ARMv7/v8 except compact instruction set) vs. variable length (x86)

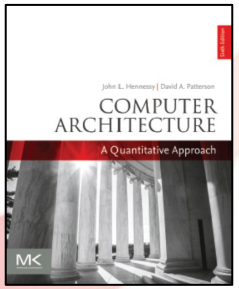


Fig. 1.5 of CA book

Instruction type/opcode	Instruction meaning
Data transfers	*Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 12-bit displacement + contents of a GPR*
`lb, lbu, sb`	Load byte, load byte unsigned, store byte (to/from integer registers)
`lh, lhu, sh`	Load half word, load half word unsigned, store half word (to/from integer registers)
`lw, lwu, sw`	Load word, load word unsigned, store word (to/from integer registers)
`ld, sd`	Load double word, store double word (to/from integer registers)
`flw, fld, fsw, fsd`	Load SP float, load DP float, store SP float, store DP float
`fmv._.x, fmv.x._`	Copy from/to integer register to/from floating-point register; "__" = S for single-precision, D for double-precision
`csrrw, csrrwi, csrrs, csrrsi, csrrc, csrrci`	Read counters and write status registers, which include counters: clock cycles, time, instructions retired
Arithmetic/logical	*Operations on integer or logical data in GPRs*
`add, addi, addw, addiw`	Add, add immediate (all immediates are 12 bits), add 32-bits only & sign-extend to 64 bits, add immediate 32-bits only
`sub, subw`	Subtract, subtract 32-bits only
`mul, mulw, mulh, mulhsu, mulhu`	Multiply, multiply 32-bits only, multiply upper half, multiply upper half signed-unsigned, multiply upper half unsigned
`div, divu, rem, remu`	Divide, divide unsigned, remainder, remainder unsigned
`divw, divuw, remw, remuw`	Divide and remainder: as previously, but divide only lower 32-bits, producing 32-bit sign-extended result
`and, andi`	And, and immediate
`or, ori, xor, xori`	Or, or immediate, exclusive or, exclusive or immediate
`lui`	Load upper immediate; loads bits 31-12 of register with immediate, then sign-extends
`auipc`	Adds immediate in bits 31–12 with zeros in lower bits to PC; used with `JALR` to transfer control to any 32-bit address
`sll, slli, srl, srli, sra, srai`	Shifts: shift left logical, right logical, right arithmetic; both variable and immediate forms
`sllw, slliw, srlw, srliw, sraw, sraiw`	Shifts: as previously, but shift lower 32-bits, producing 32-bit sign-extended result
`slt, slti, sltu, sltiu`	Set less than, set less than immediate, signed and unsigned

Fig. 5 of CA book – Cont'd

Instruction type/opcode	Instruction meaning
Control	*Conditional branches and jumps; PC-relative or through register*
`beq, bne, blt, bge, bltu, bgeu`	Branch GPR equal/not equal; less than; greater than or equal, signed and unsigned
`jal, jalr`	Jump and link: save PC + 4, target is PC-relative (`JAL`) or a register (`JALR`); if specify `x0` as destination register, then acts as a simple jump
`ecall`	Make a request to the supporting execution environment, which is usually an OS
`ebreak`	Debuggers used to cause control to be transferred back to a debugging environment
`fence, fence.i`	Synchronize threads to guarantee ordering of memory accesses; synchronize instructions and data for stores to instruction memory

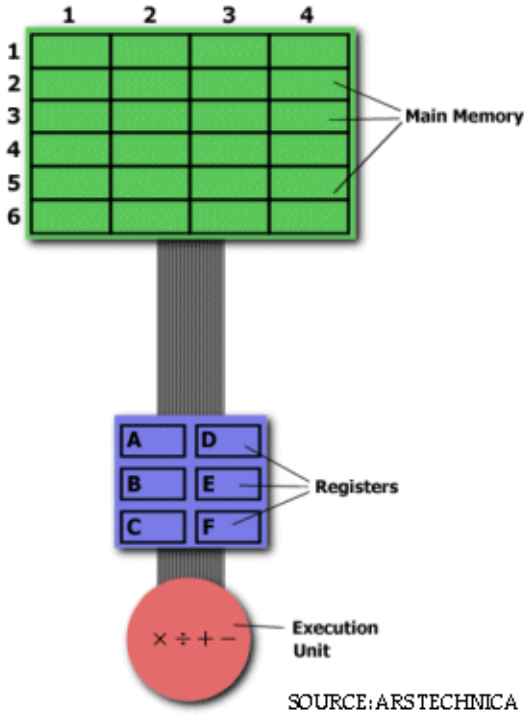
Instruction type/opcode	Instruction meaning
Floating point	*FP operations on DP and SP formats*
`fadd.d, fadd.s`	Add DP, SP numbers
`fsub.d, fsub.s`	Subtract DP, SP numbers
`fmul.d, fmul.s`	Multiply DP, SP floating point
`fmadd.d, fmadd.s, fnmadd.d, fnmadd.s`	Multiply-add DP, SP numbers; negative multiply-add DP, SP numbers
`fmsub.d, fmsub.s, fnmsub.d, fnmsub.s`	Multiply-sub DP, SP numbers; negative multiply-sub DP, SP numbers
`fdiv.d, fdiv.s`	Divide DP, SP floating point
`fsqrt.d, fsqrt.s`	Square root DP, SP floating point
`fmax.d, fmax.s, fmin.d, fmin.s`	Maximum and minimum DP, SP floating point
`fcvt._._, fcvt._._u, fcvt._u._`	Convert instructions: `FCVT.x.y` converts from type `x` to type `y`, where `x` and `y` are `L` (64-bit integer), `W` (32-bit integer), `D` (DP), or `S` (SP). Integers can be unsigned (`U`)
`feq._, flt._,fle._`	Floating-point compare between floating-point registers and record the Boolean result in integer register; "__" = `S` for single-precision, `D` for double-precision
`fclass.d, fclass.s`	Writes to integer register a 10-bit mask that indicates the class of the floating-point number ($-\infty$, $+\infty$, -0, $+0$, NaN, …)
`fsgnj._, fsgnjn._, fsgnjx._`	Sign-injection instructions that changes only the sign bit: copy sign bit from other source, the oppositive of sign bit of other source, XOR of the 2 sign bits

CISC vs. RISC

- Complex Instruction Set Computers (CISC)
  - Predecessor of RISC
  - Primary goal was to complete a task in as few lines of assembly as possible
  - The goal was achieved by building processor hardware
  - E.g.) Memory 2:3 * Memory 5:2 -> Memory 2:3

    MULT 2:3, 5:2

    (i.e., complex instruction which does not require programmer to call any loading/storing)
  - Advantages
    - Very little RAM is required to store the instructions since the length of code is relatively short
    - The compiler has to do very little work to translate a high-level language into assembly



https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/risccisc/

CISC vs. RISC – Cont'd

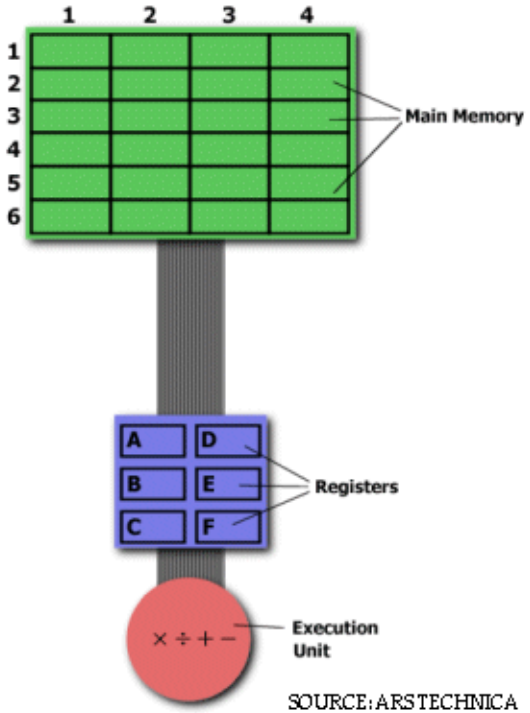
- Reduced Instruction Set Computers (RISC)
  - Use simple instructions that can be executed within one clock cycle
  - The complex instruction MULT could be divided into three simple instructions: LOAD, PROD, STORE
  - E.g.) Memory 2:3 * Memory 5:2 -> Memory 2:3

    LOAD A, 2:3

    LOAD B, 5:2

    PROD A, B

    STORE 2:3, A
  - Advantages
    - The entire program will execute in approximately the same amount of time as the instruction MULT
    - Require less transistors of hardware space
    - Because all of the instructions execute in a uniform amount of time (i.e., one clock cycle), pipelining is possible



https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/risccisc/

CISC vs. RISC – Cont'd

CISC	RISC
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: LOAD and STORE incorporated in instructions	Register to register: LOAD and STORE are independent instructions
Small code size, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

- Separating the LOAD and STORE instructions actually reduces the amount of work that the computer must perform
  - After the instruction MULT, the processor erases the registers. If data stored in the register to be re-used? Must be re-loaded
  - In RISC, the operand will remain in the register until another value is loaded in its place

CISC vs. RISC – Cont'd

- The most popular CISC processor
  - Intel x86 processor line
  - Targeting for high flexibility, but power intensive
- The most popular RISC processor
  - ARM (Advanced RISC Machine), MIPS, etc.
  - Apple M1 processor
  - Concentrated on low-power machine such as embedded processors
- Recently, the differentiations between CISC and RISC becomes meaningless
  - Intel keeps the CISC architecture externally while make it compatible with RISC externally
    - i.e., Use hardware to translate from 80x86 instructions to RISC-like instructions
  - The RISC architecture of ARM processor has evolved more toward CISC targeting for general purpose computers

RISC-V

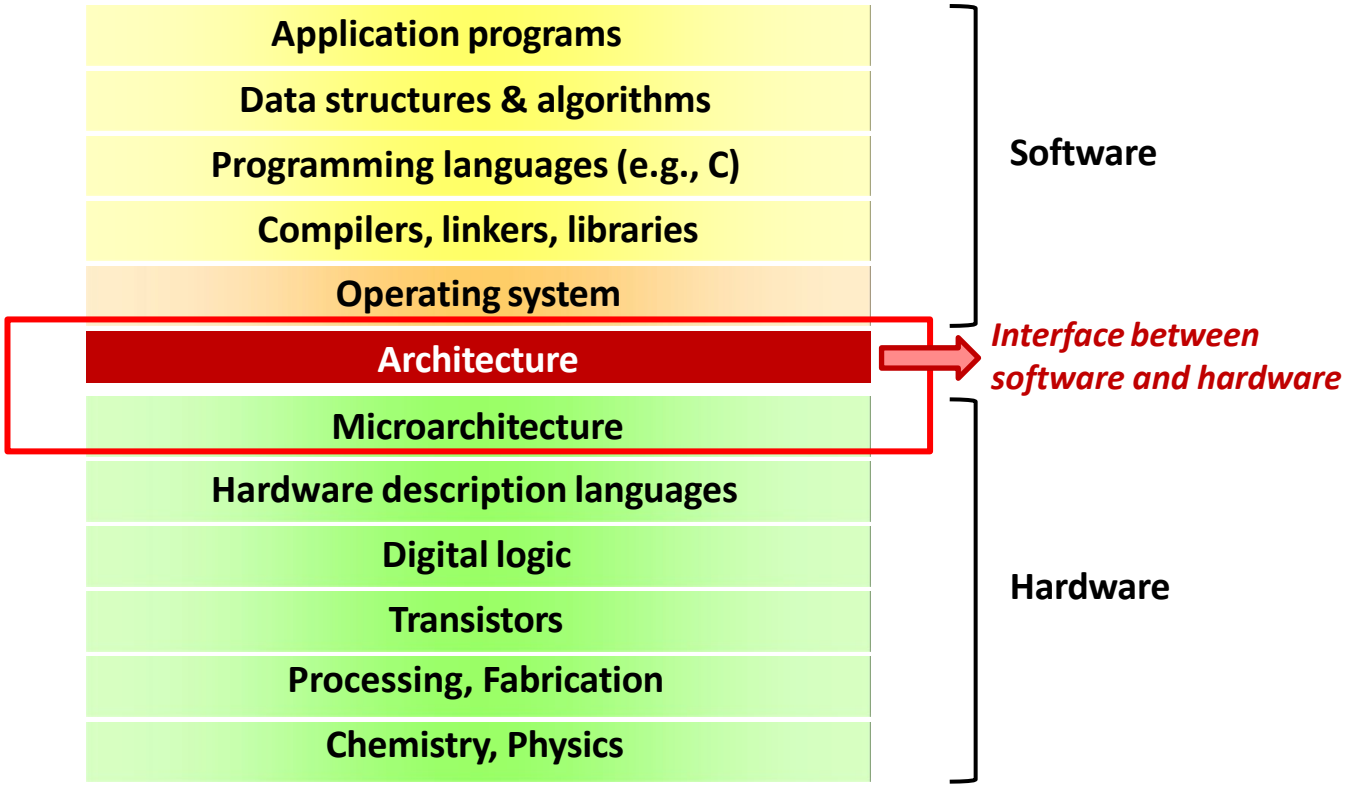
- A modern RISC instruction set developed at the UC Berkeley, 2010
- It's an open source
- Inherits its ancestors' good idea while avoiding their mistakes
  - A large set of registers, easy-to-pipeline instructions, a lean set of operations etc.
- Currently maintained by the RISC-V foundation
  - Including AMD, Google, HP enterprise, IBM, Microsoft, NVIDIA, Qualcomm, Samsung and Western Digital

Defining Computer Architecture

- "Old" view of computer architecture
  - Instruction Set Architecture (ISA) design
  - i.e. decisions regarding
    - registers, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding

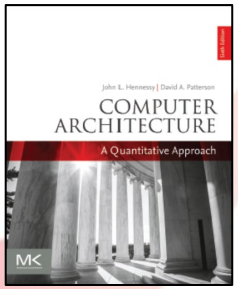
- "Real" computer architecture
  - Specific requirements of the target machine
  - Design to maximize performance within constraints: cost, power, and availability
  - Includes ISA, microarchitecture, hardware

Full levels of abstraction



Classifying instruction set architectures

(Appendix A.2 of CA book)



Classifying instruction set architectures

- How to differentiate the architectures?
  - According to the type of internal storage in a processor
    - Stack, accumulator and set of registers
  - Operands may be named implicitly or explicitly
    - A stack architecture
    - An accumulator architecture
    - A general-purpose register (GPR) architecture
      - A register-memory architecture
      - A load-store (a.k.a. register-register) architecture

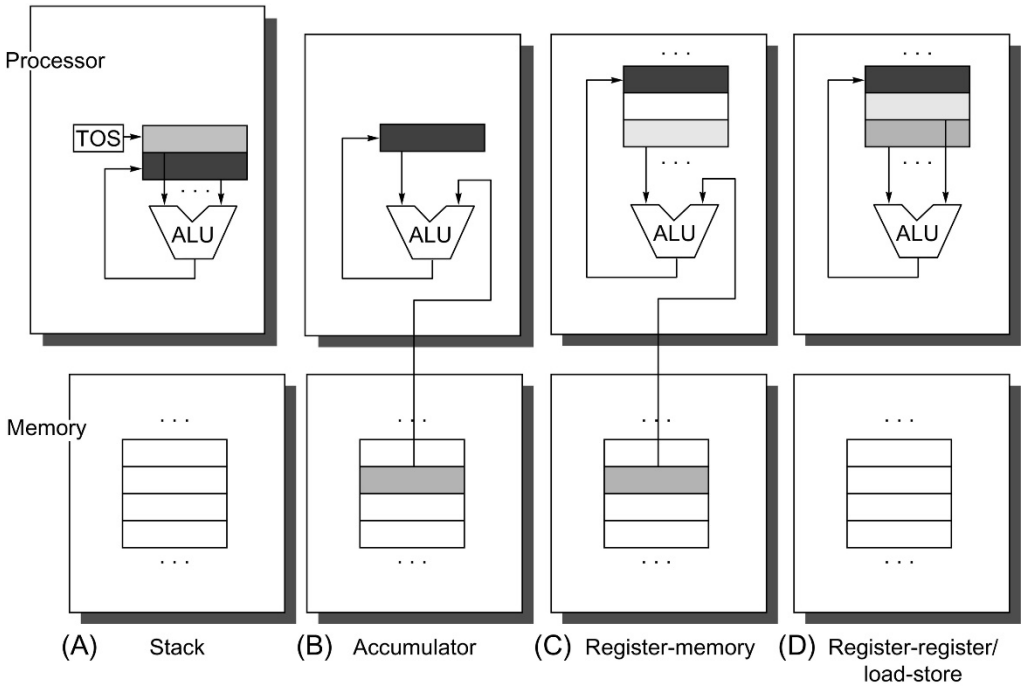
implicit

explicit

- Till 1980s, early computers used stack- and accumulator-style architectures

Classifying instruction set archit. – Cont'd

- An example of an arithmetic operation C=A+B



Implicit operands

Explicit operands

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

Classifying instruction set archit. – Cont'd

- Modern computers use the load-store architecture, because
  - Registers are faster than main memory
  - Registers are more efficient for a compiler to use than other forms of internal storage
    - What about (A*B)+(B*C)-(A*D) using stack?
  - Registers can be used to hold variables
    - The memory traffic reduces
    - The program speeds up (registers are faster than memory)
    - The code density improves (a register can be named with fewer bits than can a memory location)

Classifying instruction set archit. – Cont'd

- Two characteristics of GPR architectures
  - Whether an ALU instruction has two or three operands
    - Two-operand format – one result and one source operand
    - Three-operand format – one result and two source operand
  - How many of the operands may be memory addresses in ALU instructions
    - The number of memory operands vary from none to three
  - Combinations of these two attributes

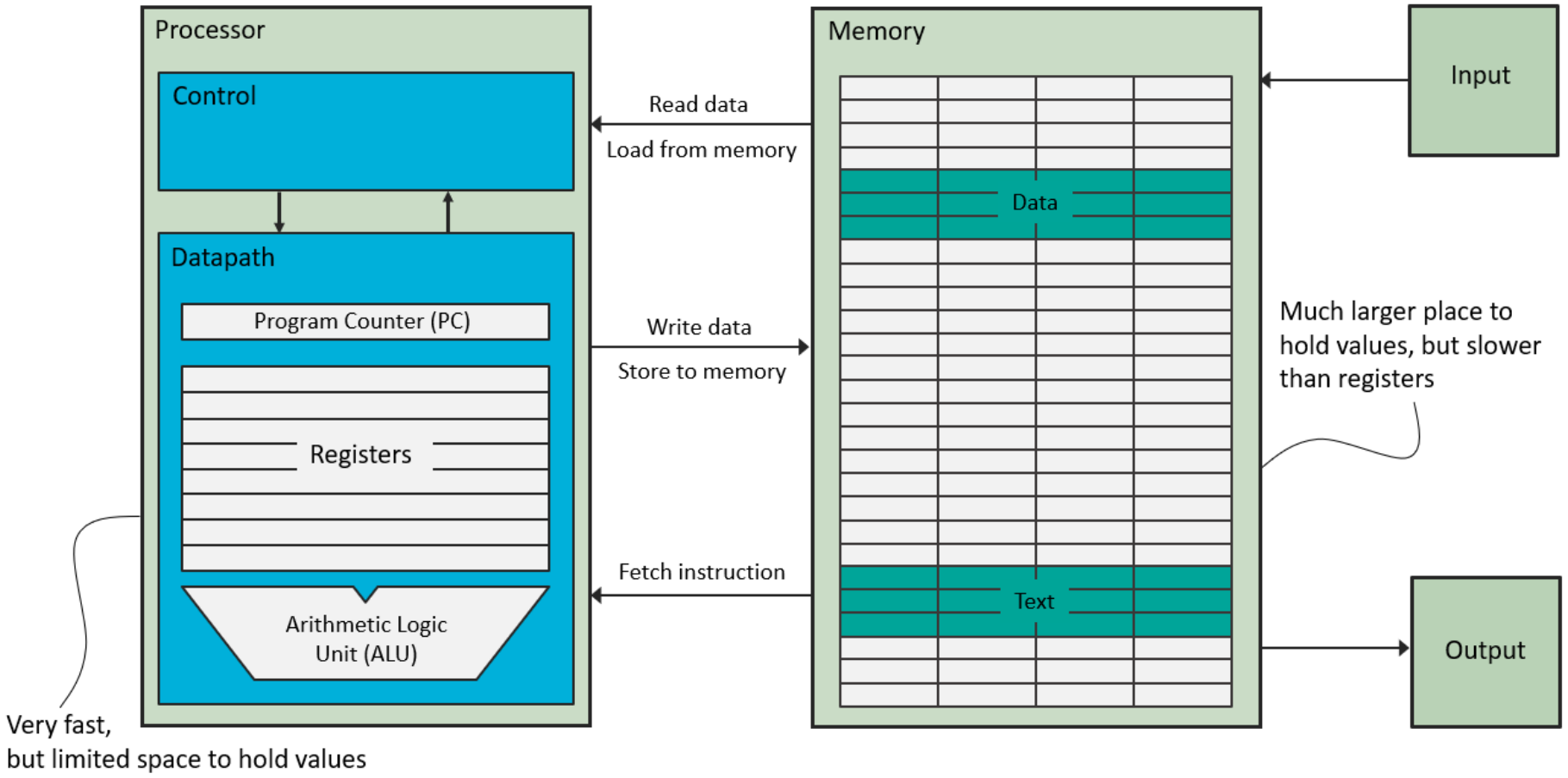
Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	ARM, MIPS, PowerPC, SPARC, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Classifying instruction set archit. – Cont'd

- The following advantages and disadvantages are not absolute. The actual impact depends on the compiler and implementation strategy

Type	Advantages	Disadvantages
Register-register (0, 3)	Simple, fixed-length instruction encoding. Simple code generation model. Instructions take similar numbers of clocks to execute (see Appendix C)	Higher instruction count than architectures with memory references in instructions. More instructions and lower instruction density lead to larger programs, which may have some instruction cache effects
Register-memory (1, 2)	Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density	Operands are not equivalent because a source operand in a binary operation is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location
Memory-memory (2, 2) or (3, 3)	Most compact. Doesn't waste registers for temporaries	Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today.)

Memory (Load-store)



Thank you!